

Plateforme d'hébergement touristique

Valorisation de l'Open Data (data.gouv.fr)

Projet de développement d'une plateforme unifiée agrégeant plus de 21 000+ hébergements certifiés en France.

Open Data

API REST

CARTOGRAPHIE

BI

PRÉSENTÉ PAR:

LOUIS-CARLOS FRANCISCO

WAHAB MOUNIROU

CHRISTSYL HODONOU

MICKAEL ANDRIEU

BELICIA MEKOULOU

PROFESSEUR:

DIATTA GAOUSSOU



Introduction & Context

Le défi en 14 jours

La France, 1^{ère} destination touristique mondiale, dispose d'un patrimoine riche mais des données Open Data dispersées et complexes.

Notre défi : développer en 14 jours une plateforme agrégeant 21 000+ hébergements certifiés, transformant des fichiers CSV bruts en une expérience utilisateur fluide.



Visualisation géographique interactive



Filtres avancés sur 45k+ hébergements



Dashboards BI et statistiques territoriales

Sommaire

1. Contexte & objectifs

Le besoin métier, les livrables et la valeur du projet.

3. Fonctionnalités & expérience

Les usages cibles, le MVP et le parcours utilisateur.

5. Qualité, perspectives & démo

Tests, sécurité, limites, suites du projet et démonstration.

2. Architecture & workflow

Le parcours de la donnée, les briques techniques et l'orchestration.

4. Organisation & diagrammes

Les rôles, la roadmap et les espaces réservés aux schémas.

Le projet en bref

Une plateforme data orientée tourisme

Le projet vise à centraliser des données ouvertes sur les hébergements touristiques français afin de les rendre consultables, comparables et exploitables par différents profils d'utilisateurs.

La valeur principale est de transformer des jeux de données dispersés en une expérience cohérente : recherche, filtrage, cartographie, API et analytique.

5

membres équipe

14j

roadmap complète

1

Sources agrégées

21 K +

hébergements

Ce que nous construisons

- Chaîne complète : collecte → traitement → stockage → exposition
- API REST pour interroger les hébergements
- Interface web : recherche, carte, fiches et filtres

Pourquoi ce projet est intéressant

- Relie data engineering, dev web et visualisation
- S'appuie sur des jeux de données réels et réutilisables
- Permet une démonstration concrète de bout en bout

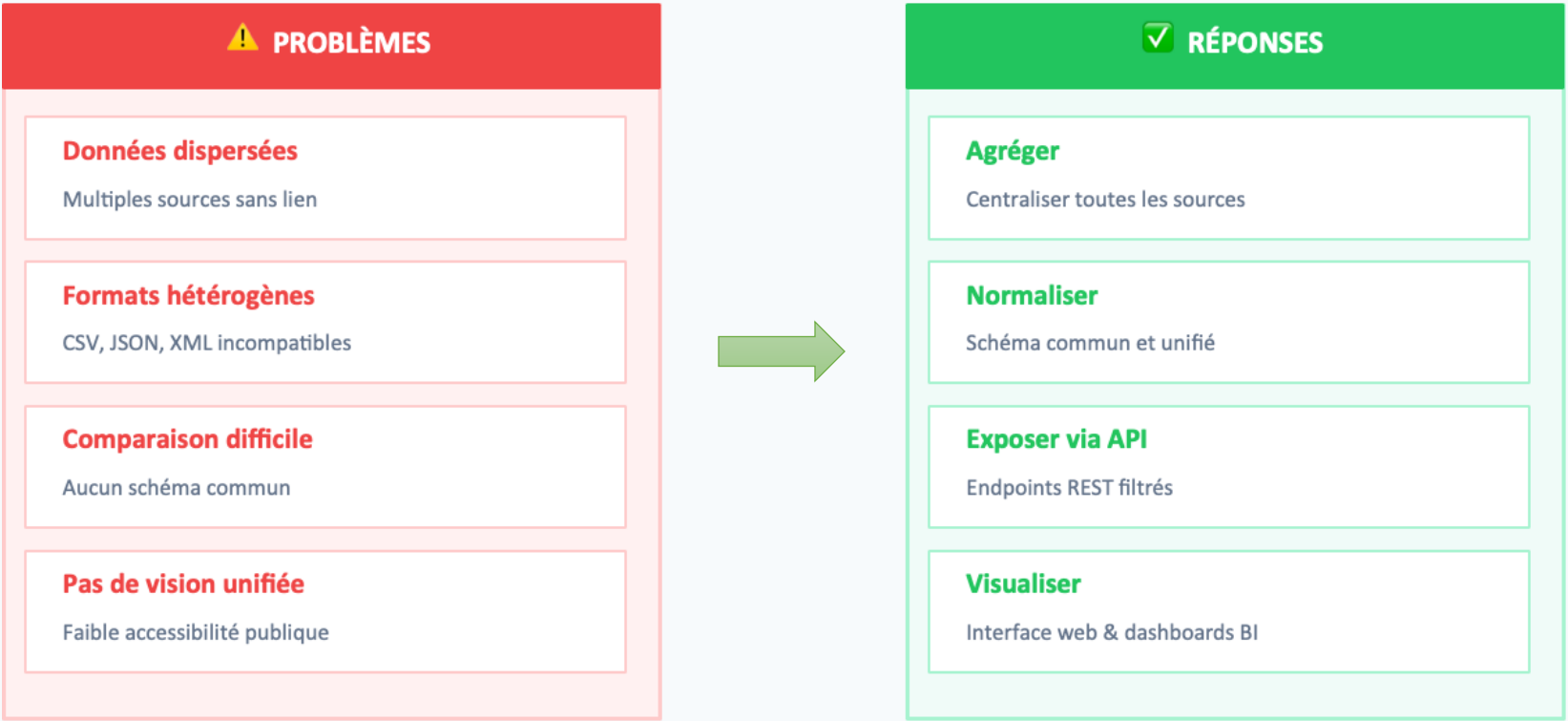
Pourquoi ce sujet ?

La France dispose d'une offre touristique vaste et variée. Pourtant, les données disponibles sont souvent éclatées entre plusieurs sources, dans des formats hétérogènes, et difficiles à comparer sans traitement préalable.

Le projet répond donc à un double enjeu : rendre ces données plus lisibles pour l'utilisateur final, et plus exploitables pour l'analyse.

Conséquences

- Recherche moins fluide
- Réutilisation analytique limitée
- Faible accessibilité pour le grand public



Objectifs et livrables attendus

OBJECTIFS TECHNIQUES



Collecter depuis data.gouv.fr (API/CSV)



Nettoyer et normaliser les informations



Stocker dans MongoDB (opérationnel)



Alimenter un DWH PostgreSQL/MariaDB



Constituer un Data Lake JSON/CSV



Développer une API REST complète



Créer un mini site web de consultation

LIVRABLES DE SOUTENANCE



Code source complet

Repository Git organisé



Base MongoDB opérationnelle

Documents normalisés



Base Data Warehouse

Tables analytiques



API fonctionnelle

Endpoints documentés



Fichiers du Data Lake

Brut + traité



Présentation soutenance

Ce support

Vision produit et valeur ajoutée

Proposition de valeur

Créer une plateforme unique qui transforme des jeux de données ouverts sur les hébergements touristiques en un service utile, lisible et réutilisable.

Pour les voyageurs

Comparer plus vite les hébergements, filtrer les résultats, visualiser une carte et consulter des fiches détaillées.

Pour les analystes

Accéder à des données consolidées, mieux exporter les résultats et produire des visualisations territoriales.

Pour les professionnels

Mieux comprendre le marché local, benchmarker l'offre et préparer des usages métier plus avancés.

En résumé

- Le projet ne se limite pas à afficher des données : il les rend exploitables.
- La plateforme connecte la donnée brute, l'usage terrain et la décision.





Voyageur / Touriste

Claire, 32 ans

Besoins clés

- Trouver rapidement un hébergement
- Comparer les options disponibles
- Visualiser l'offre sur une carte

Interface unifiée · Donnée structurée



Analyste / Chercheur

Marc, 45 ans

Besoins clés

- Accéder à des données propres
- Exporter et créer des vues analytiques
- Réutiliser l'API dans ses outils

Interface unifiée · Donnée structurée



Pro du tourisme

Sophie, 38 ans

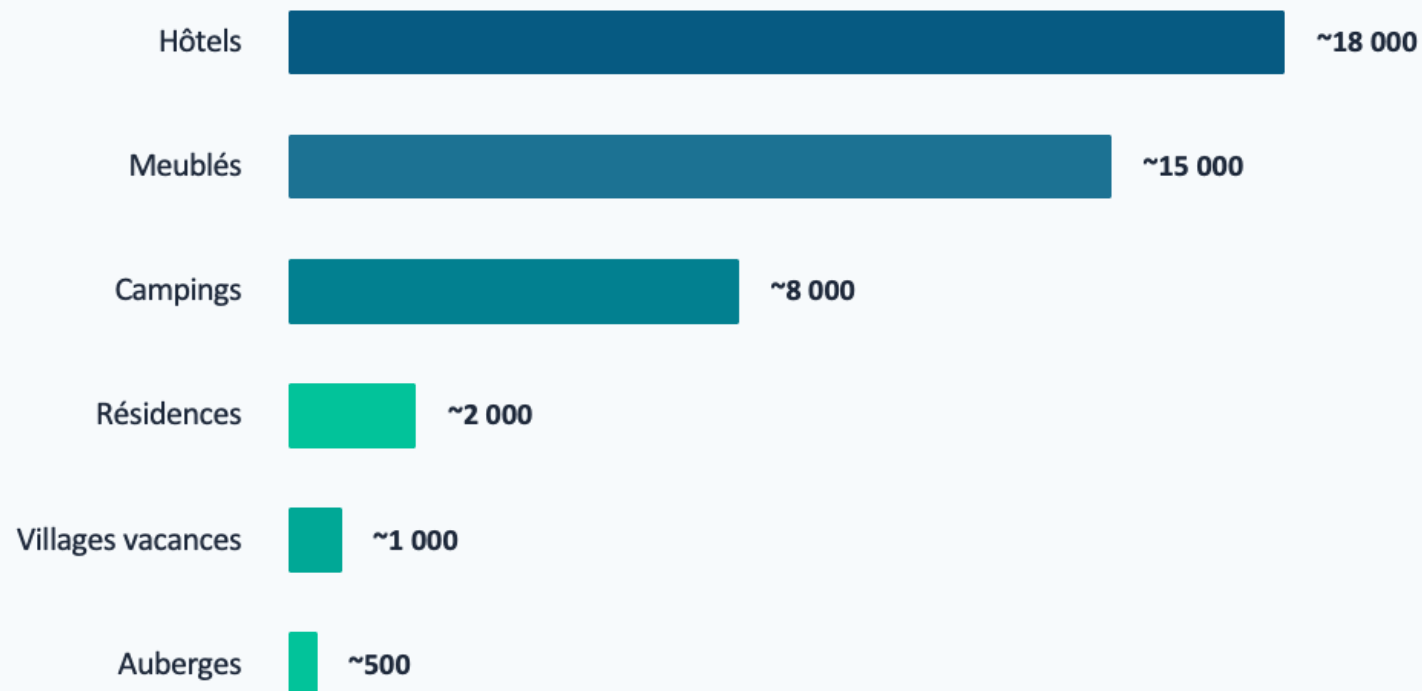
Besoins clés

- Observer la concurrence locale
- Comprendre le marché régional
- Gagner en visibilité sur l'offre

Interface unifiée · Donnée structurée

Données sources et périmètre

Répartition des hébergements



Périmètre de la version présentée

Une première base consolidée dépasse 45 000 hébergements, avec une cible produit qui peut monter plus haut à mesure que les sources sont enrichies.

21 000 +

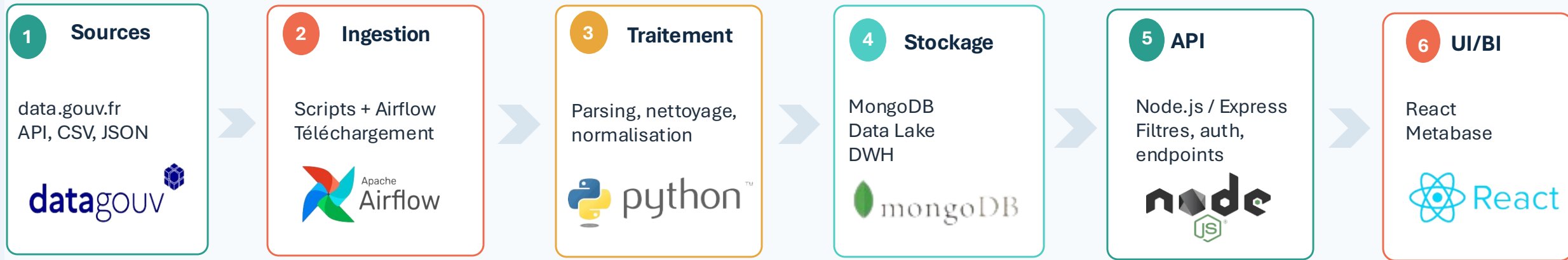
Hébergements certifiés

Sources principales

- INSEE – statistiques nationales
- UNAJ – auberges de jeunesse
- FTVAC – villages vacances
- data.gouv.fr (référentiel principal)

Workflow global du projet

WORKFLOW

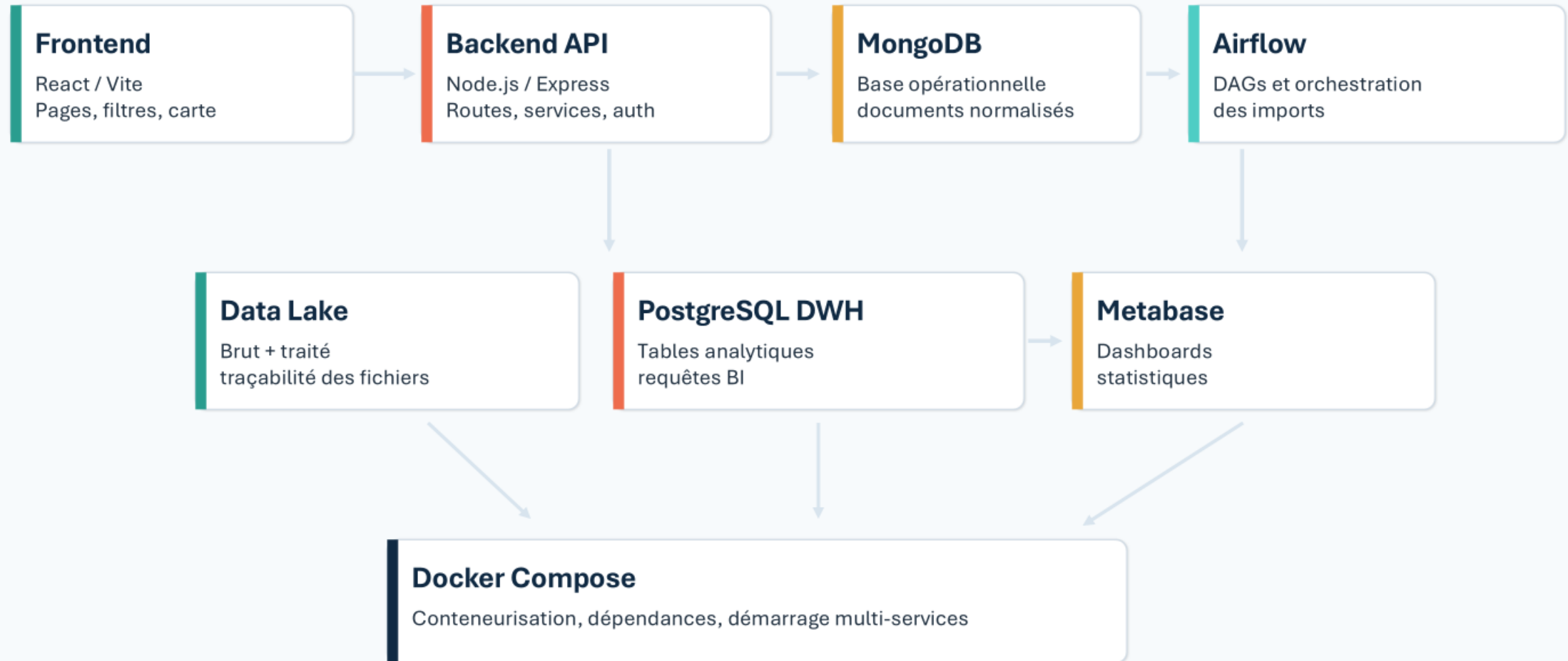


Lecture du flux :

- Pipeline data complet (ETL)
- Ingestion automatisée (Airflow)
- Transformation via scripts Python
- Stockage : MongoDB + Data Lake + DWH
- Restitution : API + UI + BI

Architecture technique globale

ARCHITECTURE



Stack technique et outils



Frontend

React 18

Vite

React Router

TailwindCSS

Leaflet

Axios



Backend

Node.js

Express 5

Mongoose

JWT

bcrypt

Rate limiting



Data / Orchestration

Python

Apache Airflow

Scripts parsing

Normalisation

ETL

Geocodage



Stockage / BI

MongoDB

PostgreSQL DWH

Data Lake JSON/CSV

Metabase



Qualité & Tests

Jest (backend)

Vitest (frontend)

Tests unitaires

Tests intégration



Déploiement

Docker

Docker Compose

Multi-services

Config isolée



Structure du frontend :

- Pages React (routing clair)
- Composants dédiés aux hébergements
- Intégration API via Axios

Fonctionnalités principales :

- Recherche et filtres
- Pagination
- Authentification
- Carte interactive (Leaflet)

Pages clés :

- Home (liste des hébergements)
- Authentification (login / register)
- Détail d'hébergement
- Routes protégées

Composants utiles :

- SearchBar
- FilterPanel
- AccommodationCard
- Sections de métadonnées

Rôle dans le workflow :

- Interface utilisateur (UI)
- Rend la donnée compréhensible
- Connecté à l'API backend
- Découplé du traitement data



Organisation backend :

- Architecture en couches : routes, controllers, services, modèles, middlewares
- Configuration Express : CORS, JSON parsing, rate limiting
- Connexion base de données
- API dédiée : authentification, hébergements, utilisateurs, favoris

Routes observées :

- /api/auth
- /api/hebergements
- /api/users
- /api/favoris

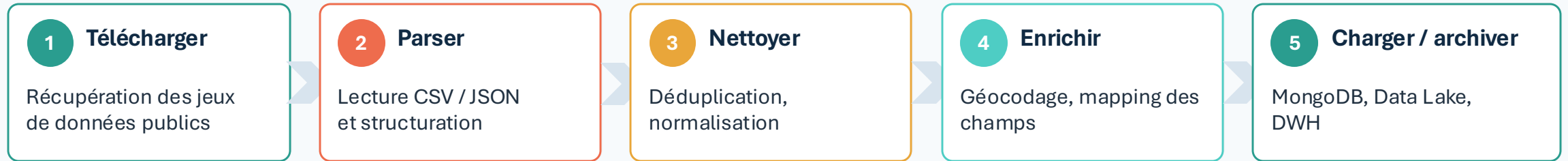
Ce que gère l'API :

- Authentification (JWT)
- Lecture et filtrage
- Pagination
- Gestion des favoris

Pourquoi cette couche est essentielle ?

- Fait le lien entre frontend et base de données
- Centralise la logique métier
- Gère la sécurité et les validations
- Expose des endpoints réutilisables

Pipeline data : ingestion et normalisation



Rôle d’Airflow :

- Orchestration des pipelines (DAGs)
- Planification des traitements
- Automatisation : import, nettoyage, archivage
- Fiabilité et reproductibilité des workflows

Pourquoi le pipeline est structuré ainsi ?

- Normalisation pour homogénéiser les données
- Conservation du brut et du traité (traçabilité)
- Séparation stockage opérationnel / analytique
- Automatisation via Airflow (pipeline reproductible)

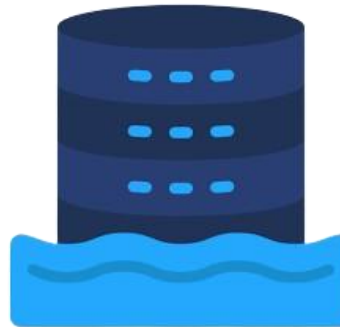
MongoDB

- Base opérationnelle
- Données normalisées
- Hébergements, utilisateurs, favoris
- Utilisée par l'application



Data Lake

- Données brutes et traitées
- Historisation des imports
- Traçabilité du pipeline
- Permet le retraitement (reprocessing)



DWH + BI

- PostgreSQL (analytique)
- Données structurées
- Requêtes et agrégations
- Visualisation via Metabase



Idée clé : une même donnée peut avoir plusieurs vies selon l'usage — opérationnel, historique et analytique.

Sécurité, qualité et tests

Sécurité

 Auth JWT · Tokens signés

 bcrypt pour les mots de passe

 CORS et rate limiting

 Réseau Docker isolé

 Validation des entrées

Qualité logicielle

 Code structuré et maintenable

 Principes SOLID respectés

 Séparation des responsabilités

 Documentation claire

 Architecture en couches

Tests

 Jest — tests backend

 Vitest — tests frontend

 Tests unitaires

 Tests d'intégration

 Vérifications API & parcours

Fonctionnalités clés du MVP

Authentification

Connexion, inscription
et gestion de session JWT

Recherche

Recherche textuelle
sur tous les hébergements

Filtres avancés

Type · Région · Classement
Capacité · Prix

Carte interactive

Visualisation géographique
via Leaflet + clusters

Fiche détail

Consultation approfondie
d'un hébergement

Favoris / BI

Sauvegarde d'intérêt
et usages analytiques

Parcours utilisateur type

UX



1

1. Rechercher

L'utilisateur saisit un mot-clé ou choisit une zone.

2

2. Filtrer

Il affine les résultats selon le type, la capacité ou le classement.

3

3. Visualiser

Les résultats sont consultables en liste et sur la carte.

4

4. Comprendre

Il ouvre une fiche détaillée pour comparer les établissements.

5

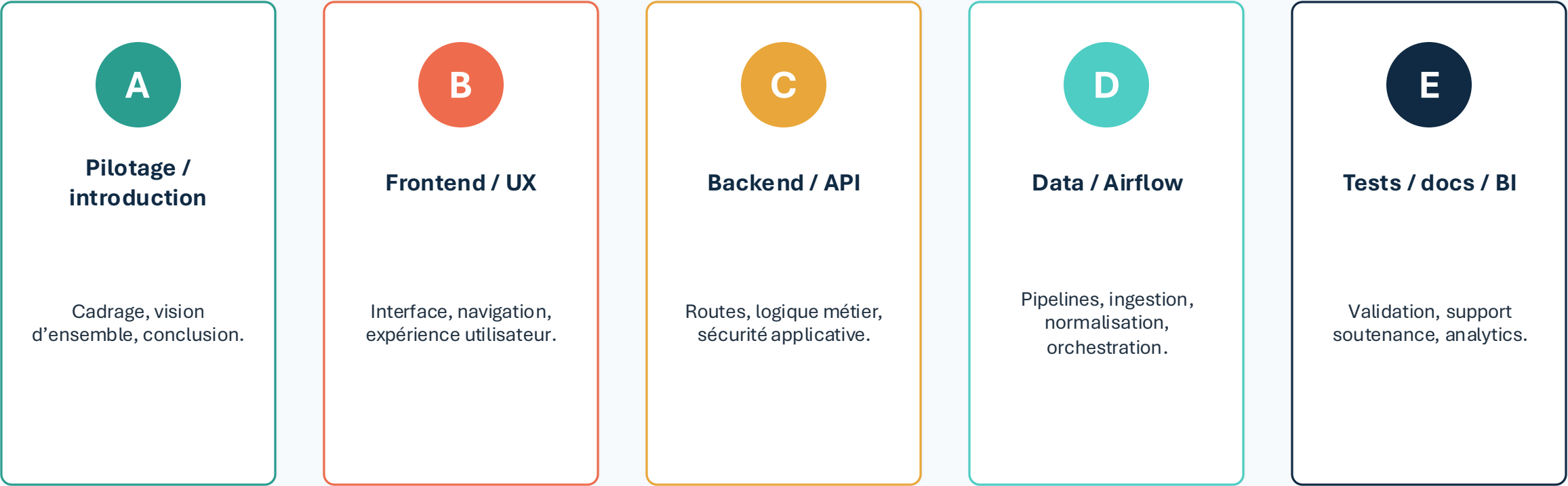
5. Exploiter

Il sauvegarde, exporte ou réutilise les données selon son profil.

Message à faire passer

Le parcours utilisateur est simple en apparence, mais il repose sur une chaîne technique complète en arrière-plan. C'est précisément ce qui fait la richesse du projet.

Équipe projet et répartition des rôles



Organisation

Rituels quotidiens, rôles clairs et traitement rapide des imprévus

Daily meeting

Priorités, objectifs, dépendances

Point d'avancement

Blocages, arbitrages, réallocation

Démo / merge

Validation, merge, actions du lendemain

Gestion des imprévus

1. Signalement dans le canal TEAMS ou au daily
2. Qualification : impact, urgence, dépendances
3. Repriorisation du backlog / décalage non-MVP
4. Arbitrage PM + référent technique
5. Traçabilité dans le board et validation du plan de reprise

Cadence de pilotage

Backlog revu en continu · buffer J10/J14 pour absorber les aléas · décision orientée MVP avant amélioration.

A

Chef de projet

Planning, coordination, arbitrage

B

Frontend / UX

Interface, carte, fiches

C

Backend / API

Auth, recherche, services

D

Data engineer

Airflow, import, normalisation

E

QA / Docs

Tests, qualité, soutenance

Tuteur : Prof

Jalons de validation : cadrage, architecture, revue de sprint et préparation de la soutenance. Son rôle est de sécuriser le périmètre et de challenger les choix techniques.

14. Product Backlog

MUST

SHOULD

COULD

Priorisation MoSCoW des user stories structurantes du MVP

Epic	User Story principale	Valeur métier	MoSCoW	Sprint	Statut
DATA-01	Importer et normaliser les datasets data.gouv.fr pour disposer d'un référentiel exploitable.	Fiabilité des données et alimentation de la plateforme	Must	S1	MVP
SEARCH-01	Rechercher et filtrer les hébergements par texte, type, zone et capacité.	Découverte rapide et ciblage des offres	Must	S1	MVP
MAP-01	Afficher les établissements sur une carte interactive avec clusters et géolocalisation.	Lecture spatiale et comparaison des localisations	Must	S1	MVP
INFO-01	Consulter une fiche détaillée avec adresse, contact, capacité et mini-carte.	Aide à la décision avant consultation / réservation	Must	S1	MVP
FAVORI-01	Sauvegarder des favoris et retrouver ses établissements.	Réengagement utilisateur	Should	S2	Amélioration
DASH-01	Visualiser des statistiques par région / département et exporter les données.	Pilotage analytique et démonstration métier	Could	S2	Amélioration

Lecture backlog

Le MVP regroupe les fonctionnalités indispensables à la consultation : data, recherche, carte, fiches. Les items S2 enrichissent l'expérience sans bloquer la démo.

Méthode MoSCoW

Must = périmètre démo minimum · Should = valeur forte après MVP · Could = bonus si la capacité le permet.

Backlog d'amélioration

Avis, favoris, analytics, tests E2E, déploiement et documentation sont planifiés en Sprint 2 selon la roadmap 14 jours.

18. Release Planning

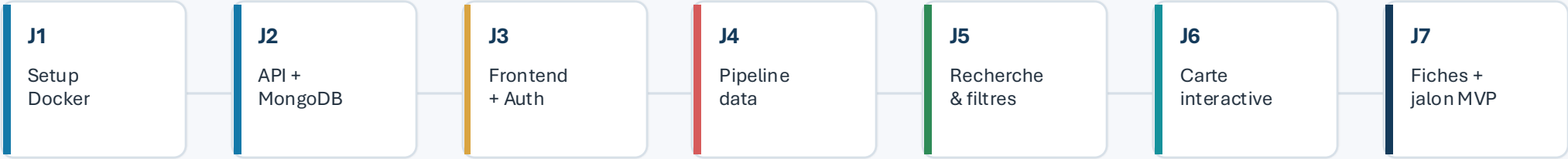
Projet planifié sur 14 jours : MVP d’abord, améliorations ensuite

Semaine 1

MVP core

Semaine 2

Finalisation / release



Ressources
A : pilotage · B : frontend · C : backend · D : data/ETL · E : QA/docs

Règle de décision
Si un aléa menace le planning, on protège le MVP et on décale les “Should / Could”.

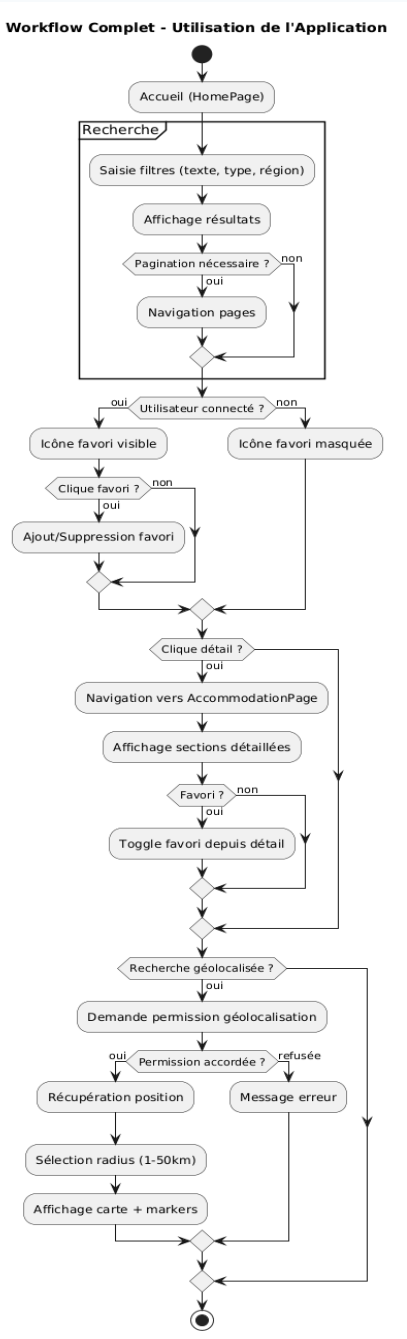
Livraison
MVP à J7, release candidate à J12, soutenance prête à J14.

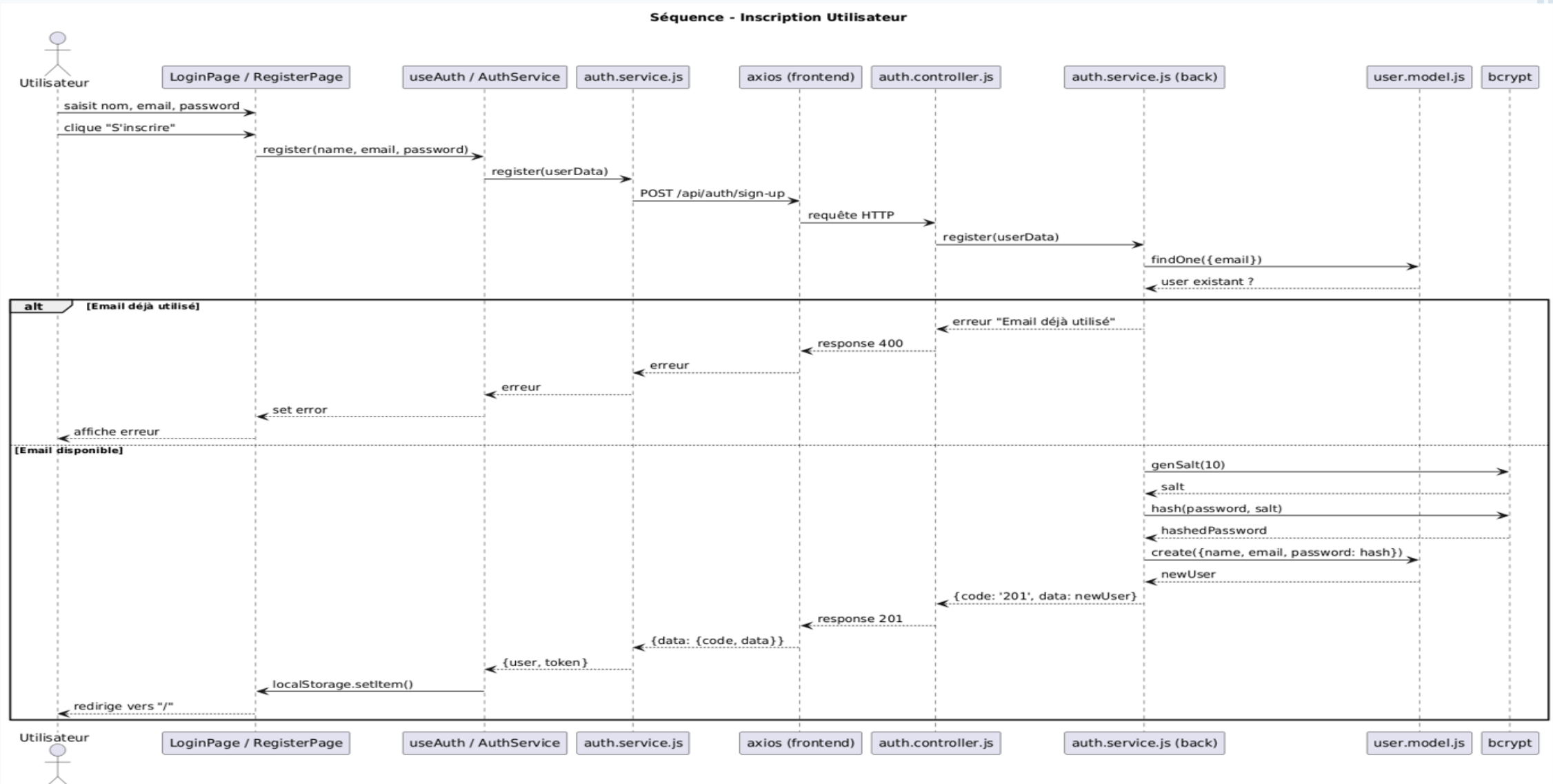


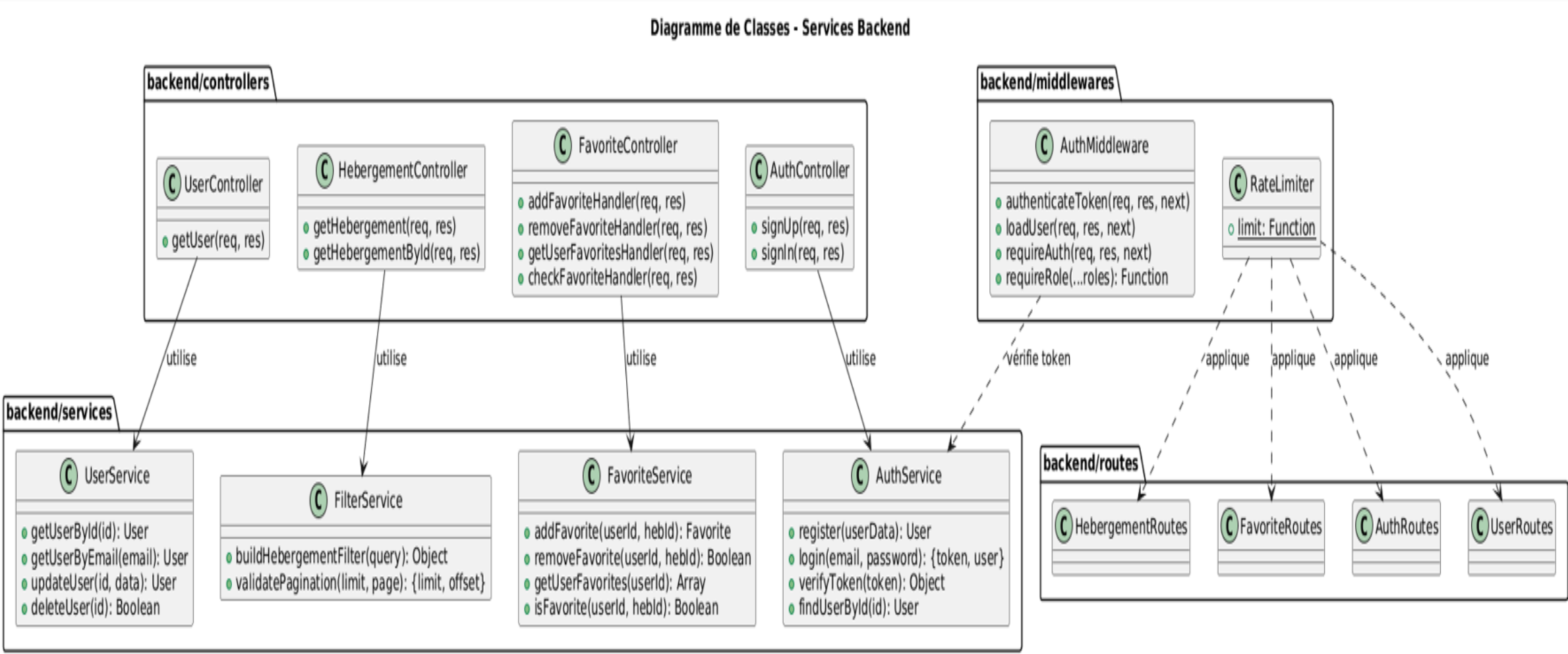


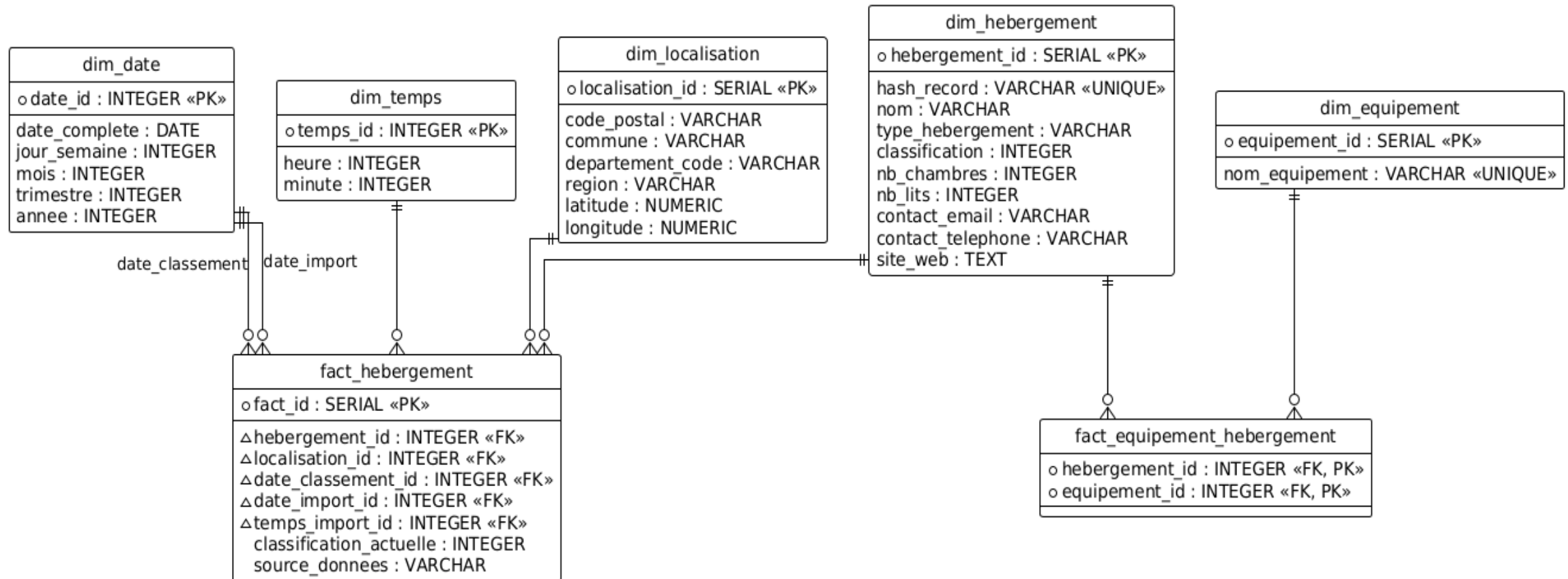
Lien Figma pour visualisation
[Lien Figma](#)

Diagramme de cas d'utilisation









Dashboard Metabase

Algo BnB

Page 1Page 2

T RégionType d'hébergementClassification minimale

Hébergements par région

Distribution par type d'hébergement

Récapitulatif par région

Region	Somme de Total Hebergements	Moyenne de Moyenne Etoiles	Somme de Capacite Lits Totale
Île-de-France	1,653	3.07	259,049
Nouvelle-Aquitaine	1,348	2.89	420,648
Provence-Alpes-Côte d'Azur	1,321	3.24	279,379
Occitanie	1,262	3.16	387,148
Auvergne-Rhône-Alpes	1,126	2.94	220,689
Bretagne	807	2.95	196,188
Pays de la Loire	651	2.98	233,480
Grand Est	497	2.8	79,535
Hauts-de-France	396	2.5	105,114
Normandie	389	3.08	71,085
Corse	345	3.1	61,445
Centre-Val de Loire	313	2.83	42,387
17 lignes			

Algo BnB

Page 1Page 2

T Région

Départements par région

Région	Code département	Nombre d'hébergements
Auvergne-Rhône-Alpes	01	24
Auvergne-Rhône-Alpes	03	54
Auvergne-Rhône-Alpes	07	131
Auvergne-Rhône-Alpes	15	33
Auvergne-Rhône-Alpes	26	54
Auvergne-Rhône-Alpes	34	9
Auvergne-Rhône-Alpes	38	109
Auvergne-Rhône-Alpes	42	44
Auvergne-Rhône-Alpes	43	22
Auvergne-Rhône-Alpes	63	112
Auvergne-Rhône-Alpes	69	118
Auvergne-Rhône-Alpes	73	166
Auvergne-Rhône-Alpes	74	250
Bourgogne-Franche-Comté	01	1
Bourgogne-Franche-Comté	21	102
		102 lignes

Défis techniques, risques et réponses

RISQUES



Hétérogénéité des sources

Créer un schéma commun et des règles de mapping.

Doublons / qualité data

Dédupliquer, contrôler et tracer les imports.

Coordonnées manquantes

Géocoder et enrichir quand c'est nécessaire.

Orchestration multi-services

Utiliser Docker Compose et Airflow.

Performance de recherche

Pagination, filtres, indexation et requêtes ciblées.

Temps limité

Prioriser le MVP et garder une roadmap lisible.

Résultats attendus et indicateurs clés

KPI

1

dataset agrégé
(Data gouv)

21k+

hébergements

< 500 ms

cible recherche

4,5 / 5

satisfaction cible

1000+

visiteurs démo



Lecture des KPI

- Couverture des données (sources agrégées)
- Richesse de la base (volume d'hébergements)
- Performance (temps de réponse)
- Expérience utilisateur (satisfaction)
- Usage du produit (visiteurs)



Limites à reconnaître

- Qualité variable selon les sources open data
- Fraîcheur des données à surveiller
- Géocodage et enrichissements perfectibles
- Certaines fonctionnalités peuvent rester au stade MVP
- Complexité croissante dès qu'on multiplie les services

Perspectives d'évolution

- Recommandation d'hébergements
- Réservation ou redirection enrichie
- Monitoring plus complet du pipeline
- Dashboards plus avancés
- Industrialisation du déploiement
- Élargissement des sources et des territoires

Conclusion

Ce que montre ce projet :

- Une vraie logique de chaîne de valeur autour de la donnée ouverte
- Une architecture complète : ingestion, traitement, stockage, API, interface
- Un sujet concret, utile et démontrable
- Une base solide pour aller vers un produit plus industrialisé



Open Data

Valoriser des jeux de données publics.

Engineering

Orchestrer, nettoyer et structurer.

Application

Exposer la donnée à des utilisateurs réels.

Perspective

Monter progressivement en maturité.

Démonstration technique



Merci pour votre attention

Questions / échanges

